10 — Installation d'un établissement

Pour le technicien qui installe G-SANTÉ dans un établissement pilote. Durée : une demi-journée, configuration comprise.

Le même code fonctionne dans les deux modes. Seul l'emplacement du serveur change.

	A. Serveur local	**B. Serveur en ligne**
Où	Un ordinateur dans l'établissement, relié aux postes par le réseau local	Un serveur chez un hébergeur
Coupure d'internet	L'établissement continue de travailler	Tout s'arrête
Coupure de courant	Le serveur doit être sur onduleur	Sans effet sur le serveur
Sauvegardes	À faire sur place, avec une copie hors du bâtiment	Chez l'hébergeur, plus une copie chez vous
Conseillé pour	CSRéf, hôpital, clinique	Cabinet bien connecté

> L'hébergement de données de santé doit respecter le cadre légal applicable. Le choix de l'hébergeur (mode B) n'est pas encore arrêté : n'installez pas de données réelles en ligne avant cette décision.

---

1. Matériel et logiciels

Serveur (mode A)

- Ordinateur dédié : 8 Go de mémoire, disque SSD de 256 Go ou plus, Linux (Ubuntu Server LTS conseillé).
- **Onduleur** obligatoire, pour que le serveur s'arrête proprement lors d'une coupure.
- Un disque externe ou une clé USB pour les sauvegardes hors site.
- Adresse IP fixe sur le réseau local (à réserver dans le routeur).

Postes de travail : n'importe quel ordinateur avec un navigateur récent (Chrome, Firefox, Edge). Rien à installer.

Imprimantes : une par poste qui imprime des tickets, ordonnances ou fiches.

Logiciels du serveur

- **PHP 8.3 ou plus**, avec les extensions `pdo_mysql`, `mbstring`, `zip`, `gd`, `intl`. Des dépendances de l'application refusent PHP 8.2.
- MariaDB 10.6 ou plus, ou MySQL 8.

- Un serveur web : Apache ou Nginx. XAMPP ne convient que si sa version embarque PHP 8.3 ou plus (le XAMPP 8.2 actuel ne suffit pas). Le plus simple est d'utiliser les paquets du système : sous Ubuntu 24.04, `apt install php8.3 php8.3-mysql php8.3-mbstring php8.3-zip php8.3-gd php8.3-intl mariadb-server apache2`.
- Composer 2, et Node.js 18 ou plus (seulement pour préparer les fichiers de l'interface).

2. Installer l'application

```
cd /var/www
git clone https://github.com/watiesnoe/g-sante.git && cd g-sante
composer install --no-dev --optimize-autoloader
npm install && npm run build
cp .env.example .env
php artisan key:generate
php artisan storage:link
```

Créer la base :

```
CREATE DATABASE gsante CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'gsante'@'localhost' IDENTIFIED BY 'un-mot-de-passe-long';
GRANT ALL PRIVILEGES ON gsante.* TO 'gsante'@'localhost';
```

N'utilisez pas le compte `root` de la base pour l'application.

3. Régler le fichier `.env`

Clé	Valeur en production
`APP_ENV`	`production`
`APP_DEBUG`	`false` (**obligatoire** : sinon une erreur affiche des données internes)
`APP_URL`	Mode A : `http://192.168.1.10` (l'IP du serveur). Mode B : `https://votre-domaine`
`DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`	Ceux créés à l'étape 2
`DB_DUMP_BINARY_PATH`	Dossier de `mysqldump` (sous XAMPP : `/opt/lampp/bin`)
`SESSION_SECURE_COOKIE`	Mode B : `true`
`BACKUP_EXTERNAL_PATH`	Point de montage du disque externe, voir l'étape 6
`BACKUP_DISKS`	`local,sauvegarde_externe` dès que le disque externe est branché
`BACKUP_ARCHIVE_PASSWORD`	Un mot de passe pour chiffrer les sauvegardes, **à noter et à mettre sous clé**
`SMS_PILOTE`	`journal` tant qu'aucun fournisseur SMS n'est choisi (les SMS sont seulement écrits dans le journal)

Puis :

```
php artisan migrate --force
php artisan db:seed --class=PermissionRoleSeeder --force
php artisan config:cache && php artisan route:cache && php artisan view:cache
```

Après toute modification de `.env`, relancez `php artisan config:cache`.

4. Sécuriser les comptes de démonstration

Le seeder crée des comptes de démonstration (`superadmin@exemple.com`, `medecin@exemple.com`, etc.) avec le mot de passe `password`. **En production, faites ceci avant toute autre chose :**

1. Connectez-vous avec `superadmin@exemple.com`.
2. Créez le compte réel du responsable informatique, avec le rôle `super_admin`.
3. Déconnectez-vous, reconnectez-vous avec ce nouveau compte.
4. **Désactivez tous les comptes `@exemple.com`** (Utilisateurs → statut).

5. Serveur web

Le site doit pointer sur le dossier `public/` de l'application, jamais sur la racine.

Droits d'écriture pour le serveur web :

```
chown -R www-data:www-data storage bootstrap/cache
```

Mode B uniquement : activez HTTPS (certificat Let's Encrypt) et bloquez l'accès direct à la base depuis l'extérieur.

Mode A : le serveur n'a pas besoin d'être visible depuis internet. N'ouvrez aucun port vers l'extérieur sur le routeur.

6. Tâches automatiques et sauvegardes

Ajoutez cette ligne à la crontab de l'utilisateur du serveur web (`crontab -u www-data -e`) :

```
* * * * * cd /var/www/g-sante && php artisan schedule:run >> /dev/null 2>&1
```

Elle déclenche :

- à 1 h, la purge des anciennes sauvegardes ;
- à 1 h 30, la sauvegarde complète ;
- à 7 h, le contrôle de santé des sauvegardes ;
- à 18 h, les rappels de rendez-vous du lendemain.

Le serveur doit donc **rester allumé la nuit**.

Disque externe : montez-le toujours au même endroit (par exemple `/mnt/sauvegarde`), renseignez `BACKUP_EXTERNAL_PATH` et `BACKUP_DISKS`, puis vérifiez :

```
php artisan backup:run --only-db
php artisan backup:list
```

Chaque semaine, échangez le disque avec un second rangé hors du bâtiment. Un vol ou un incendie ne doit pas emporter le serveur et sa sauvegarde ensemble.

Test de restauration : avant le démarrage du pilote, puis chaque trimestre, restaurez la dernière sauvegarde dans une base temporaire (procédure dans le `README.md`). Vérifiez que les derniers patients y sont.

7. Premier paramétrage

Connecté en administrateur, suivez le guide 09 — Administration, partie « Avant le démarrage du pilote » :

1. établissement, type et modules ;
2. services, salles, lits ;
3. tarifs officiels ;
4. assurances et gratuités ;
5. comptes du personnel.

8. Vérification avant d'ouvrir

- [ ] `APP_DEBUG=false`
- [ ] Comptes `@exemple.com` désactivés
- [ ] Chaque agent se connecte avec son propre compte et voit ses menus
- [ ] Un patient test suit tout le circuit : accueil, ticket, consultation, ordonnance, pharmacie, clôture de caisse. Puis annulez ses encaissements.
- [ ] Les impressions sortent sur chaque imprimante
- [ ] Une sauvegarde existe sur le disque externe et a été restaurée avec succès
- [ ] Mode A : depuis un poste, l'application s'ouvre à l'adresse `http://IP-du-serveur`
- [ ] Mode A : l'onduleur tient le serveur assez longtemps pour un arrêt propre

9. Serveur en ligne (lot 7)

Le serveur en ligne est **la même application**, installée chez un hébergeur avec `GSANTE_MODE=en_ligne`. Il ne montre rien de l'application interne : il reçoit seulement les demandes des patients et les échange avec les serveurs locaux. Il ne garde que le minimum : médecins publiés, créneaux libres, demandes de rendez-vous. **Jamais le dossier médical.**

> L'hébergeur doit être autorisé pour les données de santé. Ce choix n'est pas encore arrêté : ne mettez pas ce serveur en service avec de vrais patients avant cette décision.

Installation chez l'hébergeur : étapes 2 à 6, avec ces différences dans `.env` :

Clé	Valeur
`GSANTE_MODE`	`en_ligne`
`APP_URL`	`https://adresse-du-serveur-en-ligne` (**HTTPS obligatoire**)
`LOG_LEVEL`	`warning`, pour garder la trace des tentatives d'accès refusées

Pas de comptes de démonstration ici : n'exécutez pas le seeder des rôles.

Relier un établissement :

1. Sur le serveur local, dans Paramètres → Établissement → **5. Serveur en ligne**, relevez l'identifiant de l'établissement (un UUID).
2. Sur le serveur en ligne, enregistrez-le :

   ```
   php artisan en-ligne:site <UUID> "Nom de l'établissement"
   ```

   La commande affiche une **clé** et un **secret**. Le secret n'est affiché qu'une fois.
3. Sur le serveur local, même écran : saisissez l'adresse du serveur en ligne, la clé et le secret, puis **Synchroniser maintenant**.

4. Dans les règles de fonctionnement, cochez **« Publier les créneaux libres »** quand l'établissement est prêt à recevoir des demandes.

Fonctionnement : toutes les 5 minutes (tâche cron de l'étape 6), le serveur local récupère les demandes, envoie les décisions de l'accueil, puis publie ses créneaux libres. C'est toujours le serveur local qui appelle : **aucun port n'est à ouvrir** dans l'établissement. Chaque échange est signé avec le secret de l'établissement ; un message falsifié, rejoué ou trop ancien est refusé.

Coupure d'Internet : l'établissement continue de travailler normalement. Les décisions restent en attente et partent au retour de la connexion. L'état de chaque synchronisation est visible dans Paramètres → Établissement.

Couper l'accès d'un établissement (secret compromis, départ) :

```
php artisan en-ligne:site <UUID> "Nom" --desactiver
php artisan en-ligne:site <UUID> "Nom" --renouveler   # nouveau secret
```

Espace patient : sur le serveur en ligne, à l'adresse `/patient`. Le patient se connecte avec son numéro de téléphone et un code reçu par SMS ; il peut l'installer comme une application depuis son navigateur (« Ajouter à l'écran d'accueil »).

> Sans opérateur SMS branché (`SMS_PILOTE=journal`), les codes ne partent pas. Hors production, ils s'affichent à l'écran pour les essais ; en production, ils ne s'affichent jamais. Choisissez un fournisseur SMS avant d'ouvrir l'espace aux patients.

Essai sur un poste de développement (deux serveurs sur la même machine) :

```
scripts/serveur-en-ligne.sh installer                    # base gsante_en_ligne
scripts/serveur-en-ligne.sh relier <UUID> "Nom"           # affiche clé et secret
scripts/serveur-en-ligne.sh demarrer                      #
http://127.0.0.1:8001/patient
```

Puis, sur le serveur local : adresse `http://127.0.0.1:8001`, clé et secret dans Paramètres → Établissement → 5.

10. Mettre à jour l'application

Faites toujours une sauvegarde avant, et la mise à jour en dehors des heures de consultation.

```
php artisan backup:run --only-db
php artisan down
git pull
composer install --no-dev --optimize-autoloader
npm install && npm run build
php artisan migrate --force
php artisan config:cache && php artisan route:cache && php artisan view:cache
php artisan up
```